

Modul 1: Arbeitsblatt

Nur synthetische Daten und freigegebene Modellzugänge verwenden. Die Fallanalyse funktioniert ohne LLM. Der Kontextvergleich ist eine Review-Aufgabe, kein verpflichtender Testlauf.

1. Fallanalyse: interne Library

Auftrag: „Nutze unsere interne OrderManagementLibrary. Schreibe eine Funktion, die eine Bestellung prüft und versendet.“

Weitere Dokumentation, Typen oder Signaturen liegen nicht vor. Die Antwort bezeichnet den folgenden gekürzten Code als hypothetischen Entwurf. Nicht ausführen.

```
from OrderManagementLibrary import OrderManager # Hypothetical import

order_manager = OrderManager()
verification_result = order_manager.verify_order(order_data)
if not verification_result.get("success", False):
    ...
send_result = order_manager.send_order(order_data)
if not send_result.get("success", False):
    ...
```

6 Minuten im Pair:

- 1. Markiert gegebene Informationen, Annahmen und noch zu prüfende Aussagen.
- 2. Welche Informationen fehlen für eine Integration?
- 3. Formuliert die wichtigste Rückfrage.
- 4. Welche Belege und Prüfungen braucht ihr vor dem Einsatz?

Stelle im Entwurf	Gegeben / angenommen / zu prüfen	Benötigter Beleg
...	...	...
...	...	...
...	...	...

Was ändert die Kennzeichnung „hypothetisch“ an eurer Bewertung?

Kontrastfall für die gemeinsame Auswertung

Originalnotizen:

Project Larch was discussed on Tuesday. Maya will request a revised supplier quote. The team will revisit the launch schedule once the quote arrives.

Der zusätzliche Auftrag verlangt Verzögerungsgrund, Projektverantwortung und neuen Starttermin. Die Antwort ergänzt sinngemäß:

- Die ursprünglichen Spezifikationen passen nicht mehr.
- Maya ist projektverantwortlich.
- Der Start verschiebt sich um zwei Wochen, voraussichtlich auf Ende September 2026.

Welche Angaben stützen die Notizen? Wie würdet ihr das Update formulieren?

Quelle: vorhandener [Original-Screenshot](#). Die Ergänzungen sind hier zusammengefasst.

2. Gleiche Aufgabe, besserer Kontext

Runde 1: 3 Minuten. Runde 2: 5 Minuten. Review: 6 Minuten.

Dasselbe Modell mit denselben Einstellungen verwenden. Runde 2 in einem neuen Chat mit **Basisprompt und Ergänzung** starten. Zusätzliche Projektanweisungen, Memory und automatische Dateisuche nach Möglichkeit ausschließen, sonst als Einfluss notieren. Dies ist kein Modellbenchmark.

Basisprompt für beide Runden

```
Implementiere in C# eine reine Funktion:  
ValidationResult Validate(Order? order)  
  
Verwende diese Typen (Nullable Reference Types aktiviert,  
System.Collections.Generic verfügbar):  
  
record Order(string? CustomerNumber,  
             List<OrderItem?> Items, decimal TotalAmount);  
record OrderItem(int Quantity, decimal UnitPrice);  
record ValidationError(string Code, string Field);  
record ValidationResult(List<ValidationError> Errors,  
                       bool IsValid, bool RequiresApproval);  
  
Alle Beträge sind decimal-Werte in EUR.  
Keine externen Aufrufe. Keine Freigabeaktion auslösen.  
Validierungsfehler strukturiert zurückgeben.  
Benenne Annahmen und offene Entscheidungen ausdrücklich.  
Wenn Fachregeln fehlen, kennzeichne den Code als Entwurf.
```

Antwort speichern. Auch eine Rückfrage ist ein sinnvolles Ergebnis. Wenn kein Code entsteht, vergleicht später die erkannten Informationslücken.

Zusätzliche Fachregeln nur für Runde 2

Ergänzende Fachregeln:

- Ungültig: null-Order.
- Ungültig: CustomerNumber null, leer oder nur Whitespace.
- Ungültig: Items null oder leer; eine Position ist null.
- Ungültig: Quantity ≤ 0 oder UnitPrice < 0 .
- TotalAmount muss exakt der Summe Quantity * UnitPrice entsprechen.
- Keine Steuern, Rabatte, Rundung oder Währungsumrechnung.
- Bei TotalAmount > 10000 EUR ist zusätzliche Freigabe erforderlich.
- IsValid ist genau dann true, wenn Errors leer ist.
- RequiresApproval ist nur bei einer gültigen Bestellung true, deren TotalAmount über 10000 EUR liegt.
- Ungültige Eingaben liefern Fehler mit Code und Field. Sie dürfen keine Exception auslösen.
- Wenn eine Multiplikation oder Summe den decimal-Bereich überschreitet, liefere einen Validierungsfehler.

Erwartete Beispiele:

- K-1, eine Position (1, 20), TotalAmount 20: gültig, keine Freigabe erforderlich.
- Dieselbe Position, TotalAmount 19: ungültig.
- K-1, eine Position (1, 10000), TotalAmount 10000: gültig, keine Freigabe erforderlich.
- K-1, eine Position (1, 10000.01), TotalAmount 10000.01: gültig, Freigabe erforderlich.

Benenne verbleibende offene Entscheidungen. Behaupte keinen Testlauf, wenn du die Tests nicht tatsächlich ausgeführt hast.

Review-Matrix

Jede Zeile ist unabhängig. Gültige Basis: CustomerNumber = "K-1", Items = [new OrderItem(1, 20m)], TotalAmount = 20m.

Änderung	Erwartung in Runde 2	Runde 1: Annahme / Rückfrage	Runde 2: per Review erfüllt / verletzt / unklar
Keine	Gültig, keine Freigabe	...	...
Order null	Ungültig	...	...
Kundennummer null, leer oder Whitespace	Jeweils ungültig	...	...
Items null oder leer	Jeweils ungültig	...	...
Items enthält null	Ungültig	...	...
Quantity = 0	Ungültig	...	...
UnitPrice = -1m	Ungültig	...	...

Änderung	Erwartung in Runde 2	Runde 1: Annahme / Rückfrage	Runde 2: per Review erfüllt / verletzt / unklar
TotalAmount = 19m	Ungültig, Summenabweichung	...	...
Preis und TotalAmount = 10000m	Gültig, keine Freigabe	...	...
Preis und TotalAmount = 10000.01m	Gültig, Freigabe	...	...
Leere Items, TotalAmount = 10001m	Ungültig, RequiresApproval false	...	...
Quantity = 2, UnitPrice = decimal.MaxValue	Ungültig, keine Exception	...	...

Jeder ungültige Fall liefert `IsValid = false`, mindestens einen Fehler und `RequiresApproval = false`.

Priorisiert in den sechs Minuten: normaler Fall, null-Order, Summenabweichung, beide Freigabegrenzen und ungültige Bestellung über dem Grenzwert. Die übrigen Fälle dienen der Vertiefung.

- Welche Annahmen wurden durch Regeln ersetzt?
- Welche bekannte Regel wird weiterhin verletzt?
- Was könnt ihr durch Lesen beurteilen, was braucht Build oder Testlauf?
- Welche Entscheidungen, etwa konkrete Fehlercodes oder Fehlerreihenfolge, bleiben offen?

Unbekannte Regeln sind kein fairer Fehlermaßstab für Runde 1. Falls Runde 2 schlechter ausfällt, untersucht Regelverständnis und Umsetzung. Die Übung garantiert keine Verbesserung jeder Ausgabe.

3. Modell, Tools, Code und Mensch

Bearbeitet beide Szenarien in acht Minuten:

A: Eine unbekannte Codebasis erklären und eine vermutete Fehlerursache prüfen.

B: Ein Produktionsdeployment vorbereiten und auslösen.

Frage	A: Codebasis	B: Deployment
Was darf das Modell entscheiden?	...	...
Woher kommen belastbare Informationen?	...	...
Was übernimmt Tool oder Code?	...	...
Woran erkennen wir einen Fehler?	...	...
Welche Rechte und Freigaben sind erforderlich?	...	...
Wann wird nachgefragt oder gestoppt?	...	...

Bereitet einen strittigen Punkt für das Plenum vor.

4. Mein Transfer

- Meine wichtigste Erkenntnis: ...
- Meine nächste Änderung: ...
- Meine offene Frage: ...

Bei _____ gebe ich _____ als Kontext mit und prüfe das Ergebnis durch _____.